Is Free Self-Alignment Possible?

Anonymous Author(s)
Affiliation
Address
email

Abstract

Aligning pretrained language models (LMs) is a complex and resource-intensive process, often requiring access to large amounts of ground-truth preference data and substantial compute. Are these costs necessary? That is, *it is possible to align using only inherent model knowledge and without additional training?* We tackle this challenge with ALIGNEZ, a novel approach that uses (1) self-generated preference data and (2) representation editing to provide nearly cost-free alignment. During inference, ALIGNEZ modifies LM representations to reduce undesirable and boost desirable components using subspaces identified via self-generated preference pairs. Our experiments reveal that this nearly cost-free procedure significantly narrows the gap between base pretrained and tuned models by an average of 31.6%, observed across six datasets and three model architectures. Additionally, we explore the potential of using ALIGNEZ as a means of *expediting* more expensive alignment procedures. Our experiments show that ALIGNEZ improves DPO models tuned only using a small subset of ground-truth preference data. Lastly, we study the conditions under which improvement using ALIGNEZ is feasible, providing valuable insights into its effectiveness.

1 Introduction

Large language model (LMs) alignment involves the use of complex and expensive pipelines [1, 2, 3]. Usually at least two critical components are needed: (1) collecting human preference data, and (2) modifying pretrained model weights to better align with these preferences. Some pipelines involve more complexity (e.g., RLHF trains a reward model on the human preference data and uses it for PPO-based model optimization). Such approaches face substantial scalability challenges: collecting human preference data is costly and time-intensive, and as model sizes increase, the computational requirements for fine-tuning are likely to become prohibitive.

A prospective way to bypass the need for human preference data is to exploit knowledge ***already contained*** in the pretrained model weights. This idea is motivated by evidence suggesting that alignment merely reveals knowledge and capabilities acquired during pretraining [4, 5]. This notion has led to a growing body of literature achieving impressive results using signal contained in pretrained models for fine-tuning [6, 7, 8, 9], largely or totally sidestepping human annotation.

Next, to achieve free alignment, we must additionally obviate the need for fine-tuning. Instead, we propose to replace it with a form of ***representation editing*** that does not require computing gradients or even optimizing a proxy loss at all. Existing representation editing approaches [10, 11, 12] rely on access to ground truth data, which does not account for the unique challenges of using only signals from pretrained models. These signals are often noisier and more limited compared to human-annotated data [13, 14, 15, 16], necessitating a more tailored approach.

This work puts together these two pieces to *explore the feasibility of free self-alignment*. We align pretrained LMs to human preferences using only the knowledge from the model itself, without

Submitted to 38th Conference on Neural Information Processing Systems (NeurIPS 2024). Do not distribute.

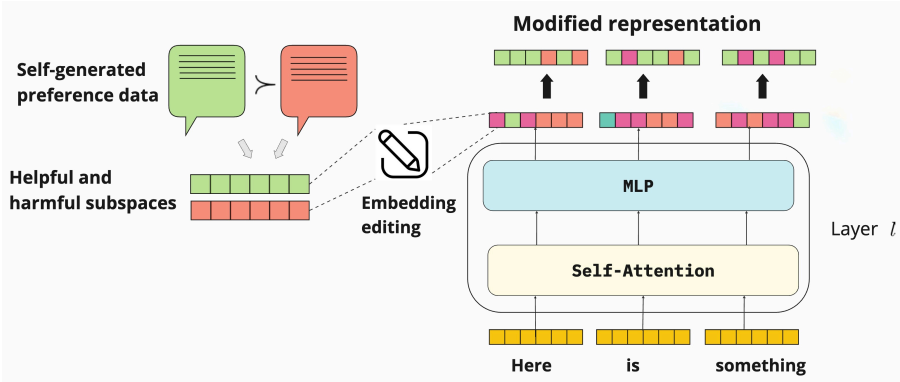

Figure 1: ALIGNEZ identifies helpful and harmful subspaces for alignment (left)—using only self-generated data. These enable modifying representations during inference (right).

additional training or fine-tuning. We introduce ALIGNEZ, a novel approach designed for this setting. Using the pretrained model's own generated preference pairs, ALIGNEZ identifies the subspaces within the model's embedding spaces that correspond to helpful and non-helpful responses. During inference, we surgically modify the model's embeddings by boosting the components from the helpful subspaces and neutralizing those from the non-helpful ones.

With this nearly cost-free procedure, we effectively narrow the performance gap between pretrained and aligned models by 31.6% across three model architectures and six datasets. Additionally, we explore the potential of ALIGNEZ to expedite more expensive alignment processes. Our experimental results demonstrate that ALIGNEZ improves upon models trained using DPO [3] with only a small subset of ground-truth preference data. In summary, our contributions include:

1. We introduce ALIGNEZ, a nearly cost-free approach that leverages preference data generated by the pretrained LM to modify its embeddings, aligning outputs to human preferences.
2. Our experiments show that ALIGNEZ significantly narrows the gap between the base model and its counterparts aligned with traditional expensive methods by 31.6% across three model architectures and six datasets.
3. We demonstrate that ALIGNEZ can *expedite* more expensive methods like DPO by improving models trained with DPO using only a small subset of ground truth preference data, by 2.2% on average.
4. We demonstrate a simple method to possibly predict conditions when free self-alignment using ALIGNEZ is possible, as a function of the quality of self-generated preference pairs.

> Our work suggests that models may be effectively steered, without additional training or supervision. Using the strategies we have developed, ***we envision the possibility of new techniques that go far beyond alignment as it exists today***, tackling such areas as fine-grained and real-time personalization, that are currently beyond the reach of existing methods.

2 Related Work

Our work tackles alignment and sits at the intersection of self-generated synthetic data and efficient model editing. We give a (necessarily) compressed introduction to these areas.

LM Alignment. The standard approach to aligning LMs with human values and preferences relies on human-annotated preference data. This data is used either to (i) train a reward function and subsequently fine-tune the LM to maximize this reward using reinforcement learning objectives, as in methods like RLHF [2, 17], or (ii) optimize a proxy loss to maximize the margin between preferred and not preferred outputs, as in methods like DPO [3]. While these methods achieve remarkable performance, they are challenging to implement due to their complex pipelines, the high cost of computing resources, and the limited scalability of acquiring human-preference data.

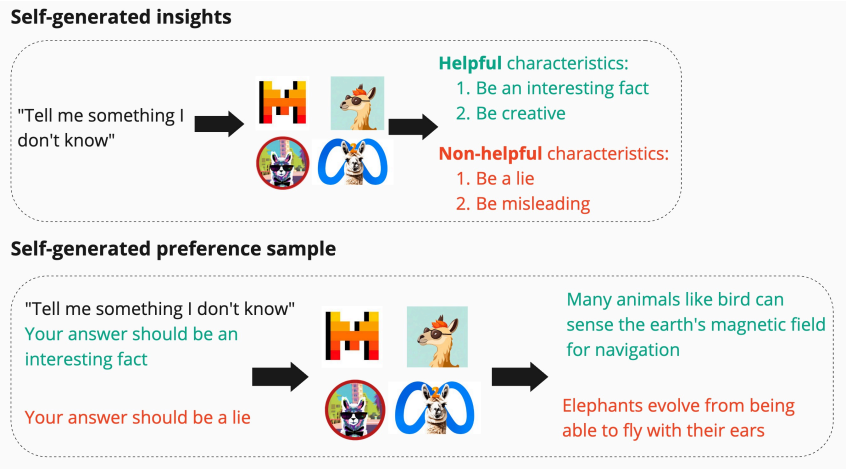

Figure 2: Generating (noisy) preference pairs. First, we prompt pretrained models to provide their *insight* on the characteristics of helpful and non-helpful responses (top). Then, we ask the model to generate responses based on these characteristics (bottom).

Self-Improvement. The difficulty of obtaining human-annotated data has led to significant efforts to bypass this requirement. Methods such as those proposed by [7, 9, 18] use manually crafted seed prompts to generate high-quality synthetic datasets from pretrained LMs, which are then used for fine-tuning or training reward models. [19] uses retrieval-augmented generation to remove reliance on manually designed prompts. Another approach, [20], leverages instruction-tuned models to assist in generating synthetic datasets. The work most similar to our approach is [6], which emphasizes *maximizing the use of knowledge from the pretrained model being aligned*. Our work takes this further by exploring whether self-alignment can be made even more cost-effective by replacing fine-tuning with representation editing, dramatically accelerating the alignment process.

Representation Editing. A parallel line of work seeks to modify model behavior without fine-tuning—doing so by solely editing the model's representations. For vision-language models like CLIP, [21] and [22] show that removing spurious or unwanted concept subspaces from embeddings boosts model accuracy on rare class predictions. [23] shows that doing so in LLM architectures reduces gender bias in generated sentences without degrading model performance in other tasks. [10, 12, 24] demonstrate that modifying embeddings during inference to steer them towards certain traits (e.g., honesty, truthfulness, sentiment) can effectively enhance these traits in the generated outputs. Similarly, [11] *learns* the appropriate embedding modification, acting as a form of fine-tuning. *These methods assume access to ground-truth* preference datasets. Our work differentiates itself by designing an intervention technique that can handle the noisier signal from synthetic data generated by LMs.

3 ALIGNEZ: (Almost) Free Alignment of Language Models

We are ready to describe the ALIGNEZ algorithm. First, we query a base pretrained LM to generate its own preference data (Figure 2). Our intuition is that, while noisy, base models have learned, from pretraining data, sufficient signal to aid in alignment. Using this self-generated data, the identify the subspaces in the LM's embedding spaces that correspond to helpful and harmful directions for alignment. During inference, we modify the LM embeddings using these identified subspaces, steering the model to generate outputs that better align with human preferences (Figure 1).

First, we describe the self-generated preference data extraction pipeline in Section 3.1. Next, we explain how ALIGNEZ identifies helpful and non-helpful subspaces in Section 3.2. Finally, we detail the embedding editing operation in Section 3.3 and the layer selection procedure for intervention in Section 3.4.

Algorithm 1: ALIGNEZ harmful and helpful subspaces identification

Parameters: base pretrained LM ω with L layers, self-generated preference pairs P^{help}, P^{harm}
for $l \in L$ **do**
  for $p_i^{help} \in P^{help}$ **do**
    Get representation at layer l: $\Phi_{i,l}^{help} \leftarrow p_i^{help}$
  end for
  Stack embedding matrix $\mathbf{H}_l^{help}$
  Identify θ_l^{help} with Equation 1
  for $p_i^{harm} \in P^{harm}$ **do**
    Get representation at layer l: $\Phi_{i,l}^{harm} \leftarrow p_i^{harm}$
  end for
  Stack embedding matrix $\mathbf{H}_l^{harm}$
  Identify θ_l^{harm} with Equation 2
end for
Returns: Helpful and harmful subspaces θ_l^{help} θ_l^{harn}

3.1 Self-generated Preference Data

First, we extract the human preference signal from the base LLM by querying it to generate its own preference data. Given a dataset D of N queries, for each query q_i, we first ask the base LM (denoted as ω) to describe characteristics of answers from a helpful agent (c_i^{help}) and a malicious agent (c_i^{harm}). Next, we pair each query with its corresponding characteristics: (c_i^{help}, q_i) and (c_i^{harm}, q_i). We then prompt the LM to generate responses conditioned on these characteristics, resulting in self-generated preference pairs for each query, denoted as (p_i^{help}, p_i^{harm}). By applying this procedure to all N samples in the dataset, we obtain self-generated preference data pairs P^{help} and P^{harm}. Note that we do not perform any prompt tuning, instead relying on a fixed set of prompt templates. This process is illustrated in Figure 2, with prompt details provided in the Appendix.

Critically, we note that the base models for generating the preference data are **not aligned or instruction-tuned**. Consequently, the resulting preference pairs may not always align with the conditioning characteristics, introducing noise into the self-preference data. To address this challenge, we tailor the embedding intervention in ALIGNEZ to accommodate this condition.

3.2 Finding Preference Directions

Next, using the noisy self-generated preference data, we identify the directions in the model embedding space that correspond with human preferences. These directions, represented as vectors $\theta \in \mathbb{R}^d$ within ω's latent space, can either (i) align with the *helpful* preferences P^{help}, facilitating alignment of the model's generated sentences, or (ii) align with the *harmful* preferences P^{harm}, leading to adverse effects on alignment [21] [25]. We denote these directions as θ^{help} and θ^{harm}, respectively. We explore several ways to identify them.

SVD-Based Identification. Our first approach for identifying these directions involves using singular value decomposition (SVD) on the preference data embeddings. We extract the first eigenvector θ. Intuitively, we view θ as the direction that best captures the underlying concepts. Let Φ_l represent the function that maps an input sentence to the LM embedding space at layer l. For each pair (p_i^{help}, p_i^{harm}), we obtain their corresponding representations $\Phi_l(p_i^{help})$ and $\Phi_l(p_i^{harm})$, which we abbreviate as $\Phi_{i,l}^{help}$ and $\Phi_{i,l}^{harm}$, respectively. To begin, we construct an embedding matrix for helpful preferences, denoted as $\mathbf{H}_l^{help}$, using these representations:

$$\mathbf{H}_l^{help} := \left[\Phi_{i,l}^{help} \middle| \ldots \middle| \Phi_{N,l}^{help}\right]^T.$$

Similarly, we create the harmful preferences embedding matrix $\mathbf{H}_l^{harm}$. Then, we proceed to identify the helpful direction as follows:

$$\begin{aligned} \mathbf{H}_l^{help} &= \mathbf{U}\Sigma\mathbf{V} \\ \theta_l^{help} &:= \mathbf{V}_{0,*}. \end{aligned} \tag{1}$$

Here, $\mathbf{U}$ and $\mathbf{V}$ represent the left and right unitary matrices produced by running SVD, respectively, and Σ is the diagonal matrix of singular values. We define θ_l^{help} as the first row of $\mathbf{V}$, corresponding to the first eigenvector of $\mathbf{H}_l^{help}$. The harmful direction θ_l^{harm} is defined similarly.

CCS-Based Identification [26]. Another approach for identifying these directions is by finding a hyperplane in the latent space that separates helpful data embeddings from harmful ones. Typically, this is achieved by training lightweight probes θ_l that maps $\Phi_{i,l}^{help}$ and $\Phi_{i,l}^{harm}$ to their respective classification labels [12]. However, we face the challenge of avoiding overfitting to the noise inherent in self-generated data, which limits the applicability of supervised classifier loss in our context. To mitigate this issue, we employ the unsupervised Contrast-Consistent Search (CCS) loss $\mathcal{L}_{CCS}$ proposed in [26]. Adapting the definition from [26] to our notations, $\mathcal{L}_{CCS}$ can be expressed as:

$$\begin{aligned} \mathcal{L}_{consistency} &:= [\theta_l(\Phi_{i,l}^{help}) - (1 - \theta_l(\Phi_{i,l}^{harm}))]^2 \\ \mathcal{L}_{confidence} &:= min\{\theta_l(\Phi_{i,l}^{help}), \theta_l(\Phi_{i,l}^{harm})\} \\ \mathcal{L}_{CCS} &:= \mathbb{E}\left[\mathcal{L}_{consistency} + \mathcal{L}_{confidence}\right]. \end{aligned} \tag{2}$$

Training θ_l with the L_{CCS} objective aims to find a separating hyperplane without fitting any labels with $\mathcal{L}_{consistency}$ and concurrently promoting maximum separation with $\mathcal{L}_{confidence}$. Unlike the SVD approach, the hyperplane identified by this method can be used as either θ_l^{harm} or θ_l^{help}, depending on which cluster it maps to class '1'. Specifically, we assign θ_l as θ_l^{harm} if it maps the majority of samples in $\mathbf{H}_l^{harm}$ to class 1.

Hybrid Identification. After exploring both methods, we find that the best results come from combining the two approaches. Specifically, we use SVD to identify θ_l^{help} and CCS to determine θ_l^{harm}. This combined approach leverages the strengths of both techniques: SVD's ability to capture the primary direction of helpful embeddings and CCS's effectiveness in identifying the hyperplane that best separates harmful embeddings from helpful ones. We describe ALIGNEZ subspace identification in Algorithm 1

3.3 Alignment with Embedding Editing.

With the harmful and helpful subspaces θ_l^{harm} and θ_l^{help} identified, we proceed to modify the LM embeddings during inference. Given x_l as the output of the MLP of layer l, the ALIGNEZ editing process proceeds as follows:

$$\hat{x}_l \leftarrow x_l - \frac{\langle x_l, \theta_l^{harm}\rangle}{\langle \theta_l^{harm}, \theta_l^{harm}\rangle}\theta_l^{harm} \quad \text{and} \quad \hat{x}_l \leftarrow \hat{x}_l + \frac{\langle \hat{x}_l, \theta_l^{help}\rangle}{\langle \theta_l^{help}, \theta_l^{help}\rangle}\theta_l^{help}.$$

In the first step, we use vector rejection to remove the influence of θ_l^{harm} from x_l. In the second step, we adjust the embedding by steering it towards the helpful direction θ_l^{help}. We perform the edit at every generation time-step. We illustrate ALIGNEZ's representation editing step in Figure 1.

3.4 Selecting Layers for Intervention.

The last piece of the puzzle is determining which layers of the LM to apply our embedding editing to. Intuitively, we want to intervene in the layers where the embeddings of X^{harm} and X^{help} *are most separable*, maximizing the effectiveness of ALIGNEZ. To accomplish this, we select the layers for intervention by identifying the subset of layers with the lowest average $\mathcal{L}_{CCS}$ loss. This ensures that our alignment interventions are targeted at the most impactful layers of the model. We provide the pseudocode for layer selection in the Appendix.

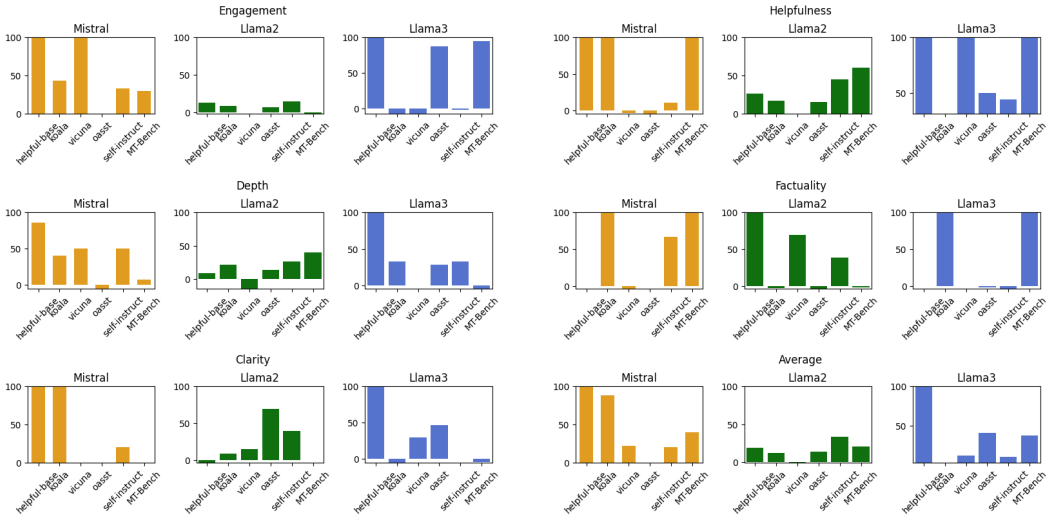

Figure 3: ALIGNEZ **Relative Improvement%**. The y-axis shows the Relative Improvement%: how much ALIGNEZ enhances the base model's performance compared to an aligned version. Values are recorded across six datasets (x-axis). A value of 100% means ALIGNEZ improves the base model to the same extent as the aligned version, while 0% means ALIGNEZ performs the same as the base model. Performance is recorded for three model families: Mistral (orange), Llama2 (green), and Llama3 (blue). We observe substantial improvements in the base models, resulting in a narrower performance gap between the base models and the aligned versions.

4 Experiments

We evaluate the following claims about ALIGNEZ.

- **Reduces alignment gap (Section 4.1).** ALIGNEZ significantly reduces the performance gap between the base model and aligned model without any additional fine-tuning and access to ground-truth preference data.
- **Expedites alignment (Section 4.2).** ALIGNEZ *expedites DPO alignment* by improving models that have been DPOed on *only a small* subset of ground-truth preference data.
- **Compatible with prompting techniques (Section 4.3).** ALIGNEZ is compatible with and can be used in combination with prompt engineering-based alignment methods [5] (Section 4.3).
- **Predicts when self-alignment is possible? (Section 4.4).** Self-generated data provides a signal about the model's ability to self-align with ALIGNEZ.

Metrics. We follow the most popular standard for automatic alignment evaluation, using GPT-4 as a judge to compare a pair of responses [27] and calculate the win rate (Win %) and lose rate (Lose %). To ensure a more nuanced and unbiased evaluation, we employ the *multi-aspect evaluation technique* proposed in [5]. Rather than evaluating the overall quality of the generated text, we ask GPT-4 to assess it across five aspects: **Engagement (E)**, **Helpfulness (H)**, **Factuality (F)**, **Depth (D)**, and **Clarity (C)**. We use the same prompt template as [5] and measure the following metrics:

1. **Net Win%** $=$ Win% $-$ Lose%: A model that produces meaningful improvement over the base model will exhibit a higher win rate than lose rate, resulting in a positive net win percentage.
2. **Relative Improvement%**.

$$\frac{\text{Net Win } \; ours - base}{\text{Net Win } \; aligned - base} \times 100.$$

This metric evaluates how much ALIGNEZ improves alignment of the base pretrained model, relative to the aligned model. A value of 0% means ALIGNEZ offers no improvement over the base model, while 100% means ALIGNEZ matches the performance of the aligned model. Positive

percentages between 0% and 100% indicate that ALIGNEZ narrows the performance gap between the base and aligned models, and a negative percentage indicates a performance decline from the base model. Excitingly, we additionally sometimes observe AlignEZ performance beyond the aligned model.

Datasets. To evaluate ALIGNEZ's generalization capability across diverse tasks and topics while keeping evaluation affordable, we use the helpfulness slice of the `just-eval-instruct` dataset [5]. This dataset is a diverse collection of queries created by sampling and merging several datasets. Specifically, we use the helpfulness slice, which combines (1) `AlpacaEval` [28] (including `helpful-base`, `koala`, `vicuna`, `open-assistant (oasst)`, and `self-instruct`), and (2) `MT-Bench` [27]. We report ALIGNEZ's performance on these individual slices.

Baselines. We compare ALIGNEZ against several base models: (1) `Mistral-7B-v0.1` [29], (2) `Llama-2-7B` [30], and (3) `Llama3-8B` [31]. As an upper bound, we also compare these base models to their aligned versions. For Llama2 and Llama3, we use `Llama-2-7b-Chat` and `Llama-3-8B-Instruct`, which are RLHF versions of the base models [30, 32]. For Mistral, we use `Mistral-7B-Instruct-v0.1`, a version of the base model fine-tuned with instruction tuning datasets [29]. We report results using the Mistral instruction-tuned model because our experiments show it outperforms the open-source Mistral DPO [33] on our evaluation datasets.

While we do not expect ALIGNEZ to consistently outperform the aligned models, we anticipate a positive **Relative Improvement%** metric. This would indicate that ALIGNEZ effectively brings the base model's performance closer to that of the aligned model without incurring additional costs.

4.1 Reducing Alignment Gap

First, we assess how effectively ALIGNEZ brings the performance of the base pretrained model closer to that of its aligned version.

Setup. All experiments use frozen LLM weights, with no additional training of these weights. We only train lightweight probes to identify θ_l^{harm} using L_{CCS} (see Section 3). Details on the hyperparameters for probe training are provided in the Appendix.

Results. Our results are shown in Figure 3. We observe consistent positive Relative Improvement% across datasets and model architectures. **This validates our claim that ALIGNEZ reduces the alignment gap between base models and their aligned versions**, occasionally even surpassing the performance of the aligned models. Remarkably, these improvements are achieved without access to ground truth preference data or any additional fine-tuning. In cases where ALIGNEZ does not yield improvements, such as with the Llama2 model on the `vicuna` dataset, we investigate the essential conditions for improvement in Section 4.4.

Figure 3 also reveals an interesting insight: ALIGNEZ shows more significant improvements in aspects like **Helpfulness** and **Factuality** compared to Engagement and Depth. This suggests that ALIGNEZ primarily enhances utility-related aspects of the base model, while its impact on stylistic aspects is comparatively limited. This indicates potential areas for further improvement in the self-generated data process. For example, generating preference data based on multiple aspects rather than a single differentiating category (e.g., helpful vs. non-helpful, as shown in Figure 2) might lead to improved overall performance.

4.2 Expediting Alignment

Next, we evaluate ALIGNEZ's ability to expedite more expensive alignment techniques like DPO. Specifically, we test whether ALIGNEZ can improve models trained with DPO using only a smaller subset of ground-truth preference data.

Setup. We perform DPO fine-tuning on the `Mistral-7b-base` model using the UltraFeedback-binarized dataset [34, 33] and do evaluation on the test set. We provide the complete DPO training parameters in the Appendix.

%	Net Win% (↑)					
	E	H	F	D	C	Avg.
1%	2.1	4.7	2.4	3.6	2.0	3.0
5%	0.0	4.6	2.1	2.3	3.5	2.4
10%	2.9	3.1	1.0	2.0	3.0	2.4
25%	0.0	0.5	2.8	-0.7	2.1	0.9

Table 1: ALIGNEZ improves DPO models trained on a small subset of the ground-truth preference dataset. The column % is the percentage of data used for DPO training.

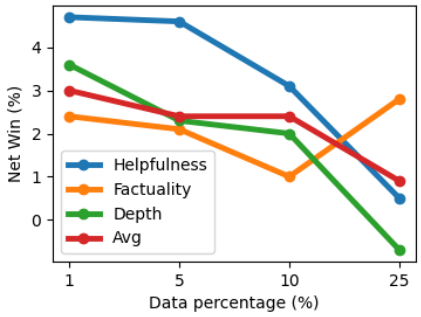

Figure 4: ALIGNEZ improvement over DPO models diminishes as we increase the training size.

Dataset	Model	Net Win% (↑)					
		E	H	F	D	C	Avg.
Vicuna	Llama2-base	10	3	3	7	10	6.6
Koala	Llama3-base	8	12	1.3	5.3	6.7	6.6

Table 2: Compatibility with prompting-based methods.

Results. Our results are shown in Table 1. ALIGNEZ enhances the alignment of models tuned using DPO on a small subset of ground truth preference data, indicated by the positive Net Win%. **This confirms our claim that ALIGNEZ expedites DPO alignment**. In Figure 4, we observe that the improvement provided by ALIGNEZ diminishes as the percentage of training data increases, which is expected since the benefit from DPO itself grows with more training data. This result highlights ALIGNEZ's potential to provide additional alignment gains when only a limited amount of ground-truth preference data is available.

4.3 Compatibility with Prompting Techniques

We also investigate the adaptability of ALIGNEZ when combined with other cost-effective alignment techniques, such as prompting [5].

Setup. We use the URIAL prompt proposed in [5] as a prefix for every query and record the performance both with and without ALIGNEZ applied. This prompt consists of manually crafted set of in-context learning examples designed to mimic the style of high-performing models such as ChatGPT and other advanced aligned LLMs.

Results. Table 2 demonstrates that ALIGNEZ enhances performance beyond what is achieved by using the prompting technique alone, as indicated by the positive Net Win%. **This confirms our claim that ALIGNEZ is compatible with prompting techniques** and shows its versatility to be used in combination with other cost-effective alignment methods.

4.4 When is Self-Alignment Possible?

We study whether the quality of self-generated data can predict if using ALIGNEZ leads to model improvement. To assess the data quality, we measure the generalization ability of classifiers trained on the self-generated data.

Setup. We train logistic regression classifiers on the embeddings of the self-generated data to predict the labels associated with the data and record the test performance. Additionally, we use an off-the-shelf sentence embedder to remove the influence of model embedding quality. The reported values are averaged across five independent runs.

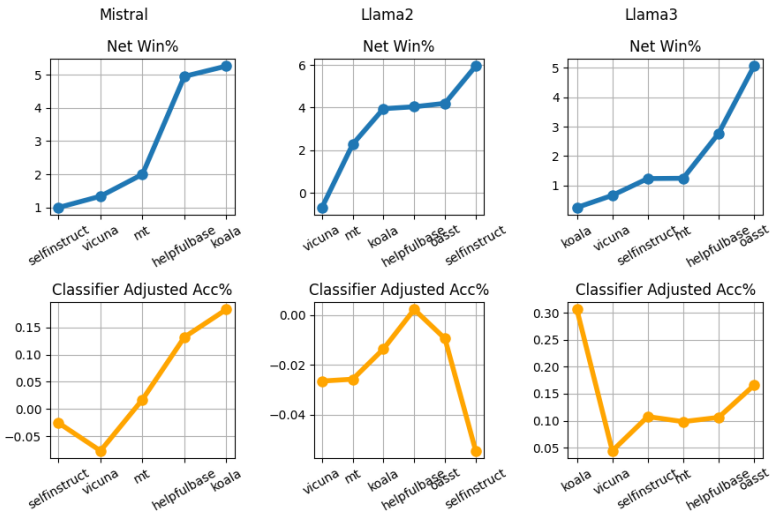

Figure 5: Net win% (blue, top row) correlation with self-generated data quality (orange, bottom row). Left to right: Mistral, Llama2, Llama3.

Results. Figure 5 shows that the average Net Win% achieved by ALIGNEZ generally correlates with the adjusted classifier accuracy. **This supports our claim that self-generated data provides a signal about the model's ability to self-align**. This correlation is particularly strong for the Mistral model. For the Llama3 and Llama2 models, the trend is mostly consistent, with some exceptions being the `koala` dataset on Llama3 (leftmost point) and the `self-instruct` dataset on Llama2 (rightmost point).

Extending this approach may offer a quick and effective method for selecting data suitable for alignment. This is crucial, as extensive research has shown that the composition and quality of training data are critical to the resulting model's performance [35, 36, 37].

5 Discussion

Limitations and Future Work. ALIGNEZ presents several limitations and avenues for future exploration. First, we perform embedding editing at every generation time step. However, it remains uncertain whether selecting specific time steps for intervention could yield further improvements. Second, while we see promising indications in Section 4.4 that the quality of self-generated data correlates with ALIGNEZ improvement, refining this characterization by developing a specialized metric for predicting the model's ability to self-align would be useful. Similarly useful would be to conduct an analysis to gauge the steerability of the base model based on the quality of its pretrained model embeddings. Additionally, our technique needs to be adapted for red-teaming scenarios, where the goal is to have the model refuse to answer certain questions instead of providing information.

Conclusion. We introduce ALIGNEZ, a novel approach for aligning pretrained LMs with human preferences without access to human-annotated data and fine-tuning. By leveraging the inherent knowledge within pretrained models, ALIGNEZ modifies model embeddings during inference to produce outputs that better align with human preferences. We empirically show that ALIGNEZ consistently enhances the alignment of the base model across multiple evaluation aspects, occasionally surpassing the performance of their aligned counterparts. Additionally, we show that ALIGNEZ can expedite more costly alignment techniques like DPO.

This work takes an initial step toward achieving truly cost-free alignment and paves the way for the development of techniques in exciting new domains like real-time dynamic alignment and fast model personalization – areas currently beyond the reach of standard alignment methods.

References

[1] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[2] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[3] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

[4] Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36, 2024.

[5] Bill Yuchen Lin, Abhilasha Ravichander, Ximing Lu, Nouha Dziri, Melanie Sclar, Khyathi Chandu, Chandra Bhagavatula, and Yejin Choi. The unlocking spell on base llms: Rethinking alignment via in-context learning. *arXiv preprint arXiv:2312.01552*, 2023.

[6] Jan-Philipp Fränken, Eric Zelikman, Rafael Rafailov, Kanishk Gandhi, Tobias Gerstenberg, and Noah D Goodman. Self-supervised alignment with mutual information: Learning to follow principles without preference labels. *arXiv preprint arXiv:2404.14313*, 2024.

[7] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[8] Zhiqing Sun, Yikang Shen, Qinhong Zhou, Hongxin Zhang, Zhenfang Chen, David Cox, Yiming Yang, and Chuang Gan. Principle-driven self-alignment of language models from scratch with minimal human supervision. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 2511–2565. Curran Associates, Inc., 2023.

[9] Zhiqing Sun, Yikang Shen, Qinhong Zhou, Hongxin Zhang, Zhenfang Chen, David Cox, Yiming Yang, and Chuang Gan. Principle-driven self-alignment of language models from scratch with minimal human supervision. *Advances in Neural Information Processing Systems*, 36, 2024.

[10] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to ai transparency, october 2023. *URL http://arxiv. org/abs/2310.01405*.

[11] Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D Manning, and Christopher Potts. Reft: Representation finetuning for language models. *arXiv preprint arXiv:2404.03592*, 2024.

[12] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. *Advances in Neural Information Processing Systems*, 36, 2024.

[13] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big?. In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623, 2021.

[14] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[15] Zachary Kenton, Tom Everitt, Laura Weidinger, Iason Gabriel, Vladimir Mikulik, and Geoffrey Irving. Alignment of language agents. *arXiv preprint arXiv:2103.14659*, 2021.

[16] Alex Tamkin, Miles Brundage, Jack Clark, and Deep Ganguli. Understanding the capabilities, limitations, and societal impact of large language models. *CoRR*, abs/2102.02503, 2021.

[17] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

[18] Timothy R McIntosh, Teo Susnjak, Tong Liu, Paul Watters, and Malka N Halgamuge. From google gemini to openai q*(q-star): A survey of reshaping the generative artificial intelligence (ai) research landscape. *arXiv preprint arXiv:2312.10868*, 2023.

[19] Hongyi Guo, Yuanshun Yao, Wei Shen, Jiaheng Wei, Xiaoying Zhang, Zhaoran Wang, and Yang Liu. Human-instruction-free llm self-alignment with limited samples. *arXiv preprint arXiv:2401.06785*, 2024.

[20] Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Luke Zettlemoyer, Omer Levy, Jason Weston, and Mike Lewis. Self-alignment with instruction backtranslation. *arXiv preprint arXiv:2308.06259*, 2023.

[21] Dyah Adila, Changho Shin, Linrong Cai, and Frederic Sala. Zero-shot robustification of zero-shot models with foundation models. *arXiv preprint arXiv:2309.04344*, 2023.

[22] Ching-Yao Chuang, Jampani Varun, Yuanzhen Li, Antonio Torralba, and Stefanie Jegelka. Debiasing vision-language models via biased prompts. *arXiv preprint 2302.00070*, 2023.

[23] Tomasz Limisiewicz, David Mareček, and Tomáš Musil. Debiasing algorithm through model adaptation. *arXiv preprint arXiv:2310.18913*, 2023.

[24] Chi Han, Jialiang Xu, Manling Li, Yi Fung, Chenkai Sun, Nan Jiang, Tarek Abdelzaher, and Heng Ji. Lm-switch: Lightweight language model conditioning in word embedding space. *arXiv preprint arXiv:2305.12798*, 2023.

[25] Fahim Dalvi, Abdul Rafae Khan, Firoj Alam, Nadir Durrani, Jia Xu, and Hassan Sajjad. Discovering latent concepts learned in bert. *arXiv preprint arXiv:2205.07237*, 2022.

[26] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022.

[27] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2024.

[28] Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Alpacaeval: An automatic evaluator of instruction-following models, 2023.

[29] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[30] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[31] AI@Meta. Llama 3 model card. 2024.

[32] Introducing Meta Llama 3: The most capable openly available LLM to date — ai.meta.com. https://ai.meta.com/blog/meta-llama-3/, 2024.

[33] Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Clémentine Fourrier, Nathan Habib, Nathan Sarrazin, Omar Sanseviero, Alexander M. Rush, and Thomas Wolf. Zephyr: Direct distillation of lm alignment, 2023.

Symbol	Definition
D	Dataset of queries
q_i	Sample query
ω	Language Model
l	Language model layer index
c_i^{help}	Characteristic of helpful answer
c_i^{help}	Characteristic of harmful/unhelpful answer
p_i^{help}	Helpful preference sample
P^{help}	Self generated helpful preference data
P^{harm}	Self generated harmful/unpreferred preference data
θ^{help}	Subspace of helpful preference samples
θ^{harm}	Subspace of harmful/unpreferred preference samples
$\Phi_{i,l}^{help}$	Embedding of p_i^{help} in layer l of ω, abbreviation of $\Phi_l(p_i^{help})$
$\Phi_{i,l}^{harm}$	Embedding of p_i^{harm} in layer l of ω, abbreviation of $\Phi_l(p_i^{harm})$
$\mathbf{H}_l^{help}$	Embedding matrix stacked from $\Phi_{i,l}^{help}$
$\mathbf{H}_l^{harm}$	Embedding matrix stacked from $\Phi_{i,l}^{harm}$
$\mathbf{V}_{0,*}$	First row of the right unitary matrix
x_l	output of MLP at layer l
$\hat{x}_l$	MLP output after ALIGNEZ embedding edit

Table 3: Glossary of variables and symbols used in this paper.

[34] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. Ultrafeedback: Boosting language models with high-quality feedback, 2023.

[35] Sang Michael Xie, Shibani Santurkar, Tengyu Ma, and Percy S Liang. Data selection for language models via importance resampling. *Advances in Neural Information Processing Systems*, 36:34201–34227, 2023.

[36] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. *arXiv preprint arXiv:2107.06499*, 2021.

[37] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. An empirical analysis of compute-optimal large language model training. *Advances in Neural Information Processing Systems*, 35:30016–30030, 2022.

[38] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.

[39] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.

[40] Sebastian Raschka. Finetuning llms with lora and qlora: Insights from hundreds of experiments, Oct 2023.

[41] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

A Appendix / supplemental material

A.1 Glossary

A.2 DPO Training details

Dataset DPO experiment were trained on binarized UltraFeedback dataset [34, 33].

Computing resources Experiment training on 1%, 5%, 10% and 25% of the dataset were run on an Amazon EC2 Instances with eight Tesla V100-SXM2-16GB GPUs.

Hyperparameters The hyperparameters we used consist of 1 training epoch, a gradient accumulation step of 1, a learning rate of $5e-5$, a max grad norm of 0.3, a warmup ratio of 0.1 (based on [38]), a precision of bfloat16, a memory saving quantize flag of "bnb.nf4", a learning rate scheduler type of cosine, and an optimizer of AdamW [39] (based on [40]). We applied PEFT [41] method to model training with hyperparameters of a r of 256, a α of 128, a dropout of 0.05 and a task type of causal language modeling (based on [38, 40]). A batch size of 16 is used to train the 1%, 5%, 10% and 25% data experiment. A batch size of 20 is used to train the full data experiment.

A.3 CCS Probe training details

We train a 1 layer linear layer with dimension of the LM embedding using the following hyperparameters: epoch = 1000, lr=1e-3, batch size=number of preference pairs, weight decay=0.01. We repeat training 10 times and take the probe with the lowest $\mathcal{L}_{CCS}$. Training is conducted in the Amazon EC2 instances with 8 Testa V100s.

A.4 ALIGNEZ Net Win and Relative Improvement Table

Table 4 shows the detailed numbers for the experiment in Section 4.1.

A.5 Layer Selection Pseudocode

Below is the pseudocode for layer selection. We select layers that have low average $\mathcal{L}_{CCS}$, by heuristically select the layers before the running mean increases significantly.

```
def select_layers(layers_loss):
    sorted_idx = np.argsort(layers_loss)
    layers_loss_sorted = layers_loss[sorted_idx]
    running_mean = []
    for i in range(1, len(sorted_idx)):
        losses = layers_loss_sorted[sorted_idx[:i]]
        running_mean.append(np.mean(losses))

    diffs = np.diff(np.array(running_mean))
    stop_edit_idx = np.argmax(diffs).flatten()[0]
    layers_to_edit = layers_loss_sorted[:stop_edit_idx]
    return layers_to_edit
```

A.6 Prompt Template

Following is the prompt template used to query the base LM to generate preference samples:

Generating helpful samples characteristics: `[QUERY]. You are a helpful assistant. Your answer to this query should:`

Generating harmful/unpreferred sample characteristics: `[QUERY]. Pretend you are a malicious and useless assistant. Your answer to this query should:`

A.7 Broader Impacts

Our work inherits the societal impacts associated with language models. On one hand, there's the risk of these models generating responses to potentially harmful queries, particularly in redteaming scenarios—an issue we acknowledge and address in our main body's limitation section. Conversely, our approach offers a potential positive societal impact by enabling a nearly cost-free alignment of language models. This capability could facilitate easier and faster alignment processes, leading to broader access to well-aligned models and ultimately contributing to positive societal outcomes.

Dataset	Model	**Net Win%** (↑)						**Relative Improvement%** (↑)					
		E	H	F	D	C	Avg.	E	H	F	D	C	Avg.
helpful-base	Mistral-7B + Ours	3	6	-2	12	6	5	-inf	-inf	-	86	-inf	-inf
	Mistral-7B-instruct	0	-16	-17	14	-12	-6						
	Llama2-7B + Ours	4	7	9	3	-3	4	13	26	112.5	9.4	-33.3	19
	Llama2-7B-chat	31	27	8	32	9	21						
	Llama3-8B + Ours	7	1	-1	4	3	2.8	150	-inf	-	-inf	100	-inf
	Llama3-8B-instruct	5	-12	-1	-6	3	-2						
koala	Mistral-7B + Ours	1.3	12	4	8	1.3	5.3	43.3	120	133.3	40	-inf	88.3
	Mistral-7B-instruct	3	10	3	20	-7	6						
	Llama2-7B + Ours	4	7	-2.6	9	2.6	4	9.3	17	-17.3	22	8.7	11.8
	Llama2-7B-chat	43	41	15	41	30	34						
	Llama3-8B + Ours	-7	5	4	4	-5	0.3	-50	31.2	200	33.3	-30	2.5
	Llama3-8B-instruct	14	16	2	12	17	12						
vicuna	Mistral-7B + Ours	7	-3	-3	10	-3	1.3	233.3	-30	-100	50	-	21.7
	Mistral-7B-instruct	3	10	3	20	-7	6						
	Llama2-7B + Ours	0	0	7	-14	3.5	-0.7	0	0	70	-42	15	-2.4
	Llama2-7B-chat	37	43	10	33	23	29						
	Llama3-8B + Ours	-7	7	0	0	3	0.7	-41	-inf	-	0	30	11.7
	Llama3-8B-instruct	17	0	0	3	10	6						
oasst	Mistral-7B + Ours	-2	-4	-2	-5	-2	-3	-200	-133	-	-100	-	-
	Mistral-7B-instruct	1	3	-5	5	-9	-1						
	Llama2-7B + Ours	3	7	-3	7	7	4.2	7.5	15.6	-50	13.5	70	14
	Llama2-7B-chat	40	45	6	52	10	30						
	Llama3-8B + Ours	7	8	-1	4	7	5	87.5	50	-11	28.6	46.7	40.3
	Llama3-8B-instruct	8	16	9	14	15	12.4						
self-instruct	Mistral-7B + Ours	1	1	2	2	1	1	33.3	11.1	66.7	50	20	20
	Mistral-7B-instruct	3	9	3	4	5	5						
	Llama2-7B + Ours	2.7	9	2.7	7.2	8	6	15	45	39	27	40	33.3
	Llama2-7B-chat	18	20	7	27	20	18						
	Llama3-8B + Ours	-2	7	-3	4	0	1.2	-14	44	-150	33.3	0	10
	Llama3-8B-instruct	14	16	2	12	17	12						
MT-Bench	Mistral-7B + Ours	3	5	0	1	0	2	30	500	-inf	7.7	0	40
	Mistral-7B-instruct	10	1	-5	13	6	5						
	Llama2-7B + Ours	-2.5	9	-1.2	6	0	2.3	-12.5	60	-24	40	0	21
	Llama2-7B-chat	20	15	5	15	1	11						
	Llama3-8B + Ours	6	5	2.5	-4	-4	1.3	95	125	-inf	-63	-400	37
	Llama3-8B-instruct	6.3	4	0	6.3	1	3.5						

Table 4: Main results table. In cases when ALIGNEZ produces positive improvement and instruct models produce zero or negative, we mark **Relative Improvement%** as -inf. In cases where both methods produce zero or negative improvement, we mark it as -.